

Micro-CFO Backend System - Technical PRD

Backend Architecture & Implementation Guide

Document Owner: Senior Backend Engineering

Last Updated: February 2026

Version: 1.0

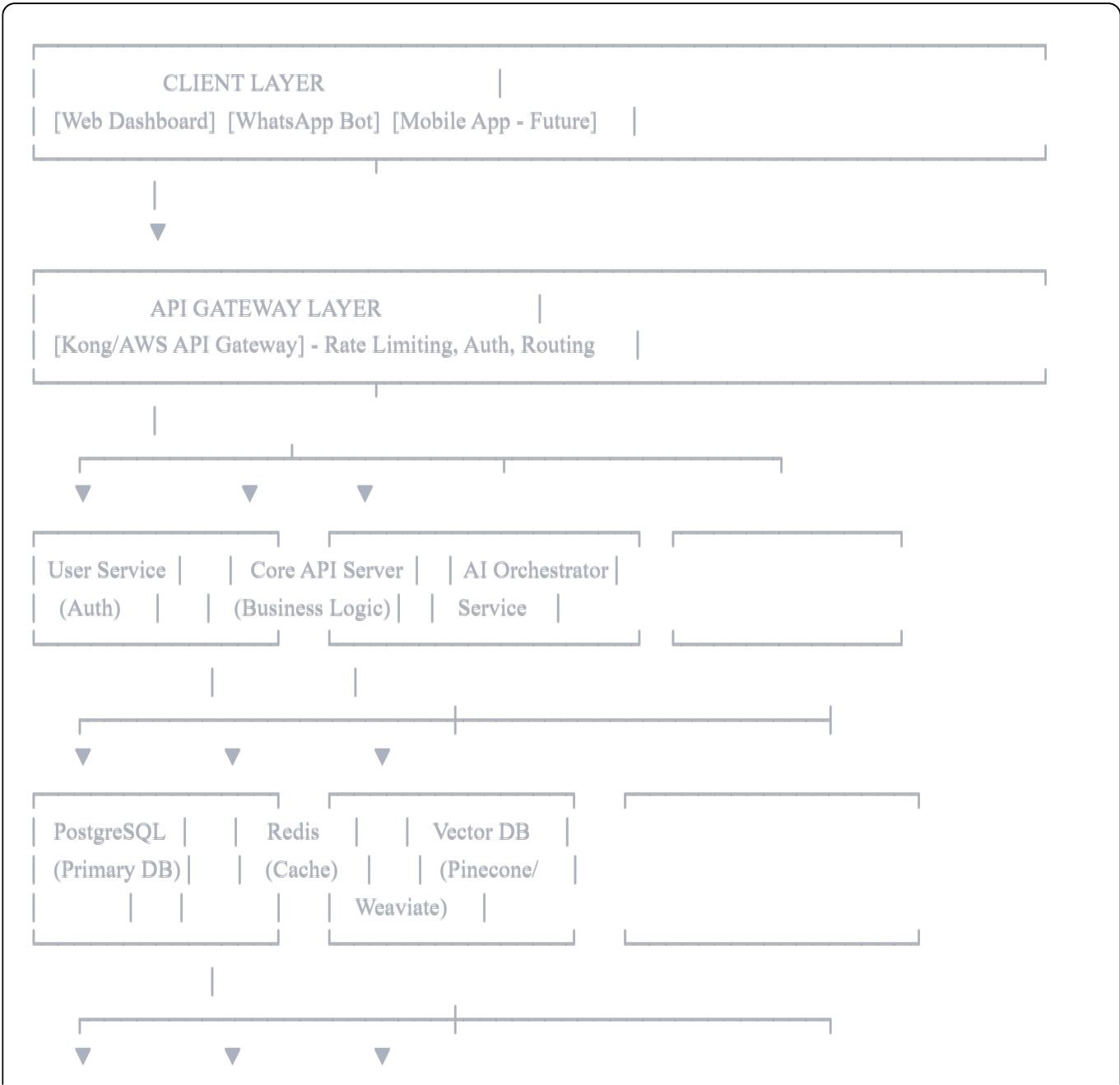
Experience Level: 10+ Years Backend Development

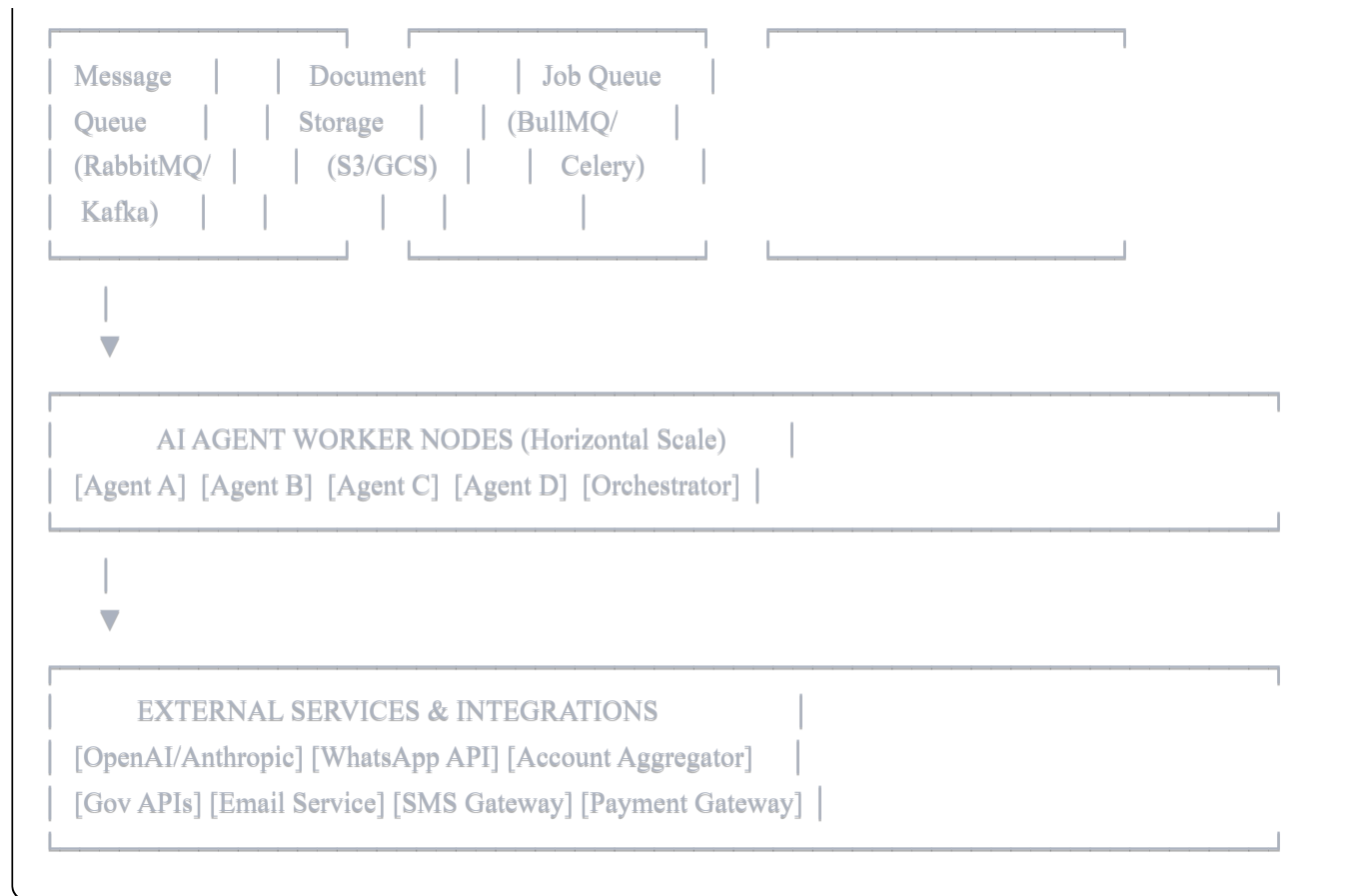
Table of Contents

1. [System Architecture Overview](#)
 2. [Technology Stack Recommendations](#)
 3. [Database Design](#)
 4. [API Design & Endpoints](#)
 5. [AI Agent Architecture](#)
 6. [Third-Party Integrations](#)
 7. [Authentication & Security](#)
 8. [Scalability & Performance](#)
 9. [Deployment & DevOps](#)
 10. [Monitoring & Observability](#)
 11. [Data Pipeline & ETL](#)
 12. [Development Timeline](#)
-

1. System Architecture Overview

1.1 High-Level Architecture





1.2 Architectural Principles

Microservices Architecture - Each major component is independently deployable:

- User/Auth Service
- Core Business Logic API
- AI Orchestrator Service
- Document Processing Service
- Notification Service
- Compliance Monitoring Service

Event-Driven Architecture - Asynchronous processing via message queues:

- Document uploaded → Queue processing → AI analysis → Results stored
- Compliance deadline → Event triggered → Notification sent
- Cash flow prediction → Alert generated → Email drafted

CQRS Pattern - Separate read and write operations:

- Write: Business transactions, document uploads
 - Read: Analytics, dashboards (can use read replicas)
-

2. Technology Stack Recommendations

2.1 Core Backend Stack

Primary Language: Python 3.11+

- **Why?** Best ML/AI library ecosystem (LangChain, OpenAI SDK, Hugging Face)
- **Framework:** FastAPI (async support, auto OpenAPI docs, excellent performance)
- **Alternative:** Node.js/TypeScript with NestJS (if team is JS-heavy)

2.2 Recommended Tech Stack

```
yaml
```

Backend Framework: FastAPI (Python) or NestJS (TypeScript)
API Protocol: REST + WebSocket (for real-time updates)
Authentication: JWT + OAuth 2.0
Primary Database: PostgreSQL 15+ (with pgvector for embeddings)
Cache Layer: Redis 7+ (with RedisJSON, RedisSearch)
Vector Database: Pinecone (managed) OR Weaviate (self-hosted)
Message Queue: RabbitMQ (simple) OR Apache Kafka (enterprise scale)
Job Queue: BullMQ (Node.js) OR Celery (Python)
File Storage: AWS S3 / Google Cloud Storage
Search Engine: Elasticsearch (optional, for advanced search)
API Gateway: Kong OR AWS API Gateway
Monitoring: Prometheus + Grafana
Logging: ELK Stack (Elasticsearch, Logstash, Kibana)
Tracing: Jaeger or Datadog APM

2.3 AI/ML Stack

yaml

LLM Provider:

- **Primary:** Anthropic Claude (Sonnet 4.5 for agents)
- **Backup:** OpenAI GPT-4 Turbo
- **Cost-effective:** Llama 3.1 (self-hosted for simple tasks)

Vision Model: GPT-4o OR Claude 3.5 Sonnet (multimodal)

Frameworks:

- **LangChain:** For agent orchestration, RAG
- **LlamaIndex:** For document indexing
- **Haystack:** Alternative RAG framework

OCR:

- Tesseract (open-source baseline)
- Google Cloud Vision API (production quality)
- AWS Textract (for structured documents)

Embedding Model:

- text-embedding-3-large (OpenAI)
- all-MiniLM-L6-v2 (open-source, fast)

3. Database Design

3.1 PostgreSQL Schema Design

Users Table

```
sql
```

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  phone_number VARCHAR(15) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE,  
  whatsapp_number VARCHAR(15),  
  business_name VARCHAR(255) NOT NULL,  
  business_type VARCHAR(100), -- Manufacturing, Services, Retail, etc.  
  gstin VARCHAR(15) UNIQUE,  
  pan_number VARCHAR(10),  
  state VARCHAR(50),  
  city VARCHAR(100),  
  industry_sector VARCHAR(100),  
  revenue_range VARCHAR(50), -- 0-1Cr, 1-10Cr, etc.  
  language_preference VARCHAR(10) DEFAULT 'en', -- en, hi, ta, te  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),  
  is_active BOOLEAN DEFAULT TRUE,  
  subscription_tier VARCHAR(50) DEFAULT 'free' -- free, basic, premium  
);  
  
CREATE INDEX idx_users_phone ON users(phone_number);  
CREATE INDEX idx_users_gstin ON users(gstin);  
CREATE INDEX idx_users_business_type ON users(business_type);
```

Documents Table

sql

```

CREATE TABLE documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  document_type VARCHAR(50) NOT NULL, -- invoice, bill, contract, etc.
  file_url TEXT NOT NULL, -- S3/GCS URL
  file_name VARCHAR(255),
  file_size_bytes BIGINT,
  mime_type VARCHAR(100),
  uploaded_via VARCHAR(50), -- web, whatsapp, email
  upload_timestamp TIMESTAMP DEFAULT NOW(),
  processing_status VARCHAR(50) DEFAULT 'pending', -- pending, processing, completed, failed
  ocr_text TEXT, -- Extracted text
  extracted_data JSONB, -- Structured data from OCR
  audit_result JSONB, -- Agent A results
  compliance_score DECIMAL(5,2), -- 0-100
  is_compliant BOOLEAN,
  flagged_issues JSONB, -- Array of issues found
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_documents_user ON documents(user_id);
CREATE INDEX idx_documents_status ON documents(processing_status);
CREATE INDEX idx_documents_type ON documents(document_type);
CREATE INDEX idx_documents_upload_time ON documents(upload_timestamp DESC);

```

Invoices Table (Structured invoice data)

```
sql
```



```

CREATE TABLE invoices (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id UUID REFERENCES documents(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  invoice_number VARCHAR(100),
  invoice_date DATE NOT NULL,
  vendor_name VARCHAR(255),
  vendor_gstin VARCHAR(15),
  customer_name VARCHAR(255),
  customer_gstin VARCHAR(15),
  subtotal DECIMAL(15,2),
  gst_amount DECIMAL(15,2),
  total_amount DECIMAL(15,2) NOT NULL,
  currency VARCHAR(3) DEFAULT 'INR',
  payment_status VARCHAR(50) DEFAULT 'pending', -- pending, paid, overdue
  payment_due_date DATE,
  invoice_type VARCHAR(50), -- purchase, sale
  category VARCHAR(100), -- raw_material, service, capital_expense, etc.
  is_personal_expense BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_invoices_user ON invoices(user_id);
CREATE INDEX idx_invoices_date ON invoices(invoice_date DESC);
CREATE INDEX idx_invoices_payment_status ON invoices(payment_status);

```

Compliance_Events Table

```
sql
```

```
CREATE TABLE compliance_events (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  event_type VARCHAR(100) NOT NULL, -- gst_filing, mca_filing, tds_return, etc.  
  due_date DATE NOT NULL,  
  title VARCHAR(255),  
  description TEXT,  
  severity VARCHAR(50), -- critical, high, medium, low  
  status VARCHAR(50) DEFAULT 'pending', -- pending, completed, missed  
  reminder_sent BOOLEAN DEFAULT FALSE,  
  applicable_law VARCHAR(100), -- GST Act, Companies Act, etc.  
  penalty_amount DECIMAL(15,2), -- Potential penalty if missed  
  source VARCHAR(100), -- Agent B, Manual, etc.  
  created_at TIMESTAMP DEFAULT NOW(),  
  completed_at TIMESTAMP  
);  
  
CREATE INDEX idx_compliance_user ON compliance_events(user_id);  
CREATE INDEX idx_compliance_due_date ON compliance_events(due_date);  
CREATE INDEX idx_compliance_status ON compliance_events(status);
```

Subsidies Table

sql

```

CREATE TABLE subsidies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  scheme_code VARCHAR(100) UNIQUE NOT NULL,
  scheme_name VARCHAR(500) NOT NULL,
  ministry VARCHAR(255),
  description TEXT,
  benefits_description TEXT,
  max_benefit_amount DECIMAL(15,2),
  eligibility_criteria JSONB, -- Structured criteria
  required_documents JSONB, -- Array of documents needed
  application_url TEXT,
  deadline DATE,
  state_specific VARCHAR(50), -- NULL for national, else state code
  sector_applicable VARCHAR(100), -- Manufacturing, Services, etc.
  embedding VECTOR(1536), -- For semantic search (pgvector)
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_subsidies_active ON subsidies(is_active);
CREATE INDEX idx_subsidies_sector ON subsidies(sector_applicable);

```

Subsidy_Matches Table

```
sql
```

```
CREATE TABLE subsidy_matches (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  subsidy_id UUID REFERENCES subsidies(id) ON DELETE CASCADE,  
  match_score DECIMAL(5,2), -- 0-100 similarity score  
  match_reason TEXT, -- Why this subsidy was matched  
  status VARCHAR(50) DEFAULT 'suggested', -- suggested, draft, applied, approved, rejected  
  application_draft JSONB, -- Auto-generated application  
  applied_date DATE,  
  result_date DATE,  
  amount_received DECIMAL(15,2),  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_subsidy_matches_user ON subsidy_matches(user_id);  
CREATE INDEX idx_subsidy_matches_status ON subsidy_matches(status);
```

Cash_Flow_Predictions Table

sql

```
CREATE TABLE cash_flow_predictions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  prediction_date DATE NOT NULL,  
  predicted_inflow DECIMAL(15,2),  
  predicted_outflow DECIMAL(15,2),  
  net_cash_flow DECIMAL(15,2),  
  confidence_score DECIMAL(5,2), -- 0-100  
  factors JSONB, -- What influenced this prediction  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_cash_flow_user_date ON cash_flow_predictions(user_id, prediction_date);
```

AI_Agent_Logs Table

sql

```
CREATE TABLE ai_agent_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  agent_name VARCHAR(50) NOT NULL, -- agent_a, agent_b, agent_c, agent_d  
  action VARCHAR(100), -- audit_invoice, find_subsidy, draft_email, etc.  
  input_data JSONB,  
  output_data JSONB,  
  execution_time_ms INTEGER,  
  tokens_used INTEGER,  
  cost_usd DECIMAL(10,6),  
  status VARCHAR(50), -- success, failed, partial  
  error_message TEXT,  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_agent_logs_user ON ai_agent_logs(user_id);  
CREATE INDEX idx_agent_logs_agent ON ai_agent_logs(agent_name);  
CREATE INDEX idx_agent_logs_created ON ai_agent_logs(created_at DESC);
```

Notifications Table

sql

```
CREATE TABLE notifications (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  notification_type VARCHAR(100), -- compliance_alert, subsidy_match, cash_flow_warning  
  title VARCHAR(255),  
  message TEXT,  
  priority VARCHAR(50), -- critical, high, medium, low  
  channel VARCHAR(50), -- whatsapp, email, sms, in_app  
  sent_at TIMESTAMP,  
  delivered_at TIMESTAMP,  
  read_at TIMESTAMP,  
  status VARCHAR(50) DEFAULT 'pending', -- pending, sent, failed  
  metadata JSONB,  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_notifications_user ON notifications(user_id);  
CREATE INDEX idx_notifications_status ON notifications(status);  
CREATE INDEX idx_notifications_created ON notifications(created_at DESC);
```

Account_Aggregator_Data Table

sql

```
CREATE TABLE account_aggregator_data (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  aa_consent_id VARCHAR(255), -- From AA framework  
  bank_name VARCHAR(255),  
  account_number_encrypted TEXT, -- Encrypted  
  account_type VARCHAR(50), -- savings, current  
  balance DECIMAL(15,2),  
  transactions JSONB, -- Last 90 days transactions  
  last_synced_at TIMESTAMP,  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_aa_data_user ON account_aggregator_data(user_id);
```

Legislative_Updates Table

sql


```
CREATE TABLE legislative_updates (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  update_type VARCHAR(100), -- gst_amendment, mca_circular, labour_law_change  
  title VARCHAR(500),  
  summary TEXT,  
  full_text TEXT,  
  source_url TEXT,  
  published_date DATE,  
  effective_date DATE,  
  applicable_sectors JSONB, -- Array of sectors  
  impact_level VARCHAR(50), -- high, medium, low  
  embedding VECTOR(1536), -- For semantic search  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_legislative_updates_date ON legislative_updates(published_date DESC);
```

3.2 Redis Cache Structure

```
python  
  
# Cache Keys Design  
user_session: {user_id} -> {JWT token data} [TTL: 24h]  
user_profile: {user_id} -> {User object JSON} [TTL: 1h]  
document_processing: {document_id} -> {Processing status} [TTL: 30min]  
subsidy_matches: {user_id} -> {Array of subsidy IDs} [TTL: 24h]  
compliance_calendar: {user_id}:{month} -> {Events array} [TTL: 6h]  
rate_limit: {user_id}:{endpoint} -> {Request count} [TTL: 1min]
```

3.3 Vector Database Schema (Pinecone/Weaviate)

Subsidy Embeddings Index

python

```
{
  "id": "subsidy_123",
  "vector": [0.123, 0.456, ...], # 1536-dim embedding
  "metadata": {
    "scheme_name": "PMFME Subsidy",
    "sector": "Manufacturing",
    "max_benefit": 100000,
    "state": "Maharashtra"
  }
}
```

Legislative Document Embeddings Index

python

```
{
  "id": "law_doc_456",
  "vector": [0.789, 0.012, ...],
  "metadata": {
    "document_type": "GST Circular",
    "date": "2024-01-15",
    "applicable_sectors": ["All"],
    "summary": "New GST rules..."
  }
}
```

4. API Design & Endpoints

4.1 API Architecture Principles

- **RESTful design** with proper HTTP verbs
- **Versioning**: `/api/v1/` prefix
- **Pagination**: Cursor-based for large datasets
- **Rate limiting**: 100 req/min for free tier, 1000 req/min for premium
- **Response format**: Consistent JSON structure

```
json
{
  "status": "success|error",
  "data": {...},
  "message": "Human-readable message",
  "metadata": {
    "timestamp": "ISO8601",
    "request_id": "uuid"
  }
}
```

4.2 Core API Endpoints

Authentication & User Management

```
http
```

POST /api/v1/auth/register

Body: {

```
"phone_number": "+919876543210",  
"business_name": "ABC Manufacturing",  
"gsttin": "27XXXXX1234X1Z5",  
"language": "en"
```

}

Response: {

```
"user_id": "uuid",  
"otp_sent": true
```

}

POST /api/v1/auth/verify-otp

Body: {

```
"phone_number": "+919876543210",  
"otp": "123456"
```

}

Response: {

```
"access_token": "jwt_token",  
"refresh_token": "refresh_token",  
"user": {...}
```

}

POST /api/v1/auth/refresh

Body: {"refresh_token": "..."}
Response: {"access_token": "new_jwt"}

GET /api/v1/users/me

Headers: Authorization: Bearer {token}

Response: {User profile object}

PATCH /api/v1/users/me

Body: {"business_name": "New Name", ...}

Response: {Updated user object}

Document Management

http

POST /api/v1/documents/upload

Headers: Authorization: Bearer {token}

Content-Type: multipart/form-data

Body: {

```
"file": <binary>,  
"document_type": "invoice",  
"uploaded_via": "web"
```

}

Response: {

```
"document_id": "uuid",  
"status": "processing",  
"estimated_completion": "30 seconds"
```

}

GET /api/v1/documents/{document_id}

Response: {

```
"id": "uuid",  
"file_url": "https://...",  
"processing_status": "completed",  
"audit_result": {  
  "is_compliant": false,  
  "issues": ["Personal expense detected", "Missing GST number"],  
  "confidence": 0.95
```

```
},
```

```
"extracted_data": {...}
```

}

GET /api/v1/documents

Query: ?user_id={uuid}&page=1&limit=20&status=completed

Response: {

```
"documents": [...],
```

```
"pagination": {
```

```
"total": 150,  
"page": 1,  
"limit": 20,  
"next_cursor": "..."  
}  
}
```

DELETE /api/v1/documents/{document_id}

Response: {"status": "deleted"}

Invoice Management

http

GET /api/v1/invoices

Query: ?status=overdue&start_date=2024-01-01&end_date=2024-12-31

Response: {

 "invoices": [...],

 "total_amount": 500000,

 "overdue_count": 5

}

POST /api/v1/invoices

Body: {

 "document_id": "uuid", # Optional

 "invoice_number": "INV-001",

 "vendor_name": "XYZ Suppliers",

 "total_amount": 50000,

 "invoice_date": "2024-01-15",

 ...

}

Response: {Invoice object}

PATCH /api/v1/invoices/{invoice_id}

Body: {"payment_status": "paid"}

Response: {Updated invoice}

Compliance Monitoring

http

GET /api/v1/compliance/calendar

Query: ?month=2024-02

Response: {

 "events": [

 {

 "id": "uuid",

 "event_type": "gst_filing",

 "due_date": "2024-02-20",

 "title": "GSTR-3B Filing",

 "severity": "critical",

 "status": "pending"

 },

 ...

]

}

GET /api/v1/compliance/risk-score

Response: {

 "overall_score": 75,

 "breakdown": {

 "gst_compliance": 80,

 "mca_compliance": 70,

 "labour_compliance": 75

 },

 "pending_tasks": 3,

 "potential_penalties": 25000

}

POST /api/v1/compliance/mark-completed

Body: {"event_id": "uuid"}

Response: {"status": "completed"}

Subsidy Discovery & Management

http

GET /api/v1/subsidies/matches

Response: {

```
"matches": [  
  {  
    "subsidy_id": "uuid",  
    "scheme_name": "PMFME Scheme",  
    "match_score": 95,  
    "max_benefit": 100000,  
    "deadline": "2024-06-30",  
    "status": "suggested"  
  },  
  ...  
]  
}
```

POST /api/v1/subsidies/generate-application

Body: {

```
"subsidy_id": "uuid"  
}
```

Response: {

```
"application_id": "uuid",  
"draft_content": {...}, # Pre-filled form data  
"required_documents": ["GST Certificate", "Bank Statement"],  
"status": "draft"  
}
```

POST /api/v1/subsidies/submit-application

Body: {

```
"application_id": "uuid",  
"additional_documents": ["doc_id_1", "doc_id_2"]  
}
```

Response: {

```
"status": "submitted",  
  "tracking_number": "SUBSIDY-2024-001"  
}
```

GET /api/v1/subsidies/applications

Response: {
 "applications": [...],
 "total_applied": 5,
 "total_approved": 2,
 "amount_received": 150000
}

Cash Flow Management

http

GET /api/v1/cashflow/forecast

Query: ?days=90

Response: {

```
"predictions": [  
  {  
    "date": "2024-02-15",  
    "predicted_inflow": 100000,  
    "predicted_outflow": 80000,  
    "net_cash_flow": 20000,  
    "confidence": 0.85  
  },  
  ...  
],  
"critical_dates": [  
  {  
    "date": "2024-03-01",  
    "issue": "Cash shortage predicted",  
    "shortfall_amount": 50000  
  }  
]  
}
```

GET /api/v1/cashflow/overdue-invoices

Response: {

```
"invoices": [  
  {  
    "invoice_id": "uuid",  
    "customer_name": "ABC Corp",  
    "amount": 50000,  
    "days_overdue": 15,  
    "suggested_action": "Send reminder email"  
  },  
  ...  
]
```

```
    ...
  ]
}

POST /api/v1/cashflow/draft-collection-email
Body: {
  "invoice_id": "uuid",
  "tone": "polite" # polite, firm, urgent
}
Response: {
  "email_draft": {
    "subject": "Payment Reminder: Invoice INV-001",
    "body": "Dear ...",
    "to": "customer@example.com"
  }
}

POST /api/v1/cashflow/send-email
Body: {
  "email_draft_id": "uuid",
  "custom_edits": {...}
}
Response: {
  "status": "sent",
  "sent_at": "2024-02-05T10:30:00Z"
}
```

AI Agent Monitoring

http

```
GET /api/v1/agents/activity
```

```
Query: ?agent=agent_a&days=7
```

```
Response: {  
  "agent_name": "Visual Auditor",  
  "total_executions": 150,  
  "success_rate": 0.95,  
  "avg_execution_time_ms": 3500,  
  "total_cost_usd": 2.50,  
  "recent_actions": [...]  
}
```

```
GET /api/v1/agents/logs
```

```
Query: ?user_id={uuid}&limit=50
```

```
Response: {  
  "logs": [  
    {  
      "timestamp": "2024-02-05T10:00:00Z",  
      "agent": "agent_c",  
      "action": "find_subsidy",  
      "result": "3 matches found",  
      "execution_time_ms": 2000  
    },  
    ...  
  ]  
}
```

Notifications

http

GET /api/v1/notifications

Query: ?status=unread&limit=20

Response: {

```
"notifications": [  
  {  
    "id": "uuid",  
    "type": "compliance_alert",  
    "title": "GST Filing Due in 3 Days",  
    "message": "...",  
    "priority": "high",  
    "created_at": "...",  
    "read_at": null  
  },  
  ...  
],  
"unread_count": 5  
}
```

PATCH /api/v1/notifications/{notification_id}/read

Response: {"status": "marked_as_read"}

POST /api/v1/notifications/preferences

Body: {

```
"whatsapp_enabled": true,  
"email_enabled": true,  
"sms_enabled": false,  
"notification_types": {  
  "compliance_alerts": "all",  
  "subsidy_matches": "high_priority_only",  
  "cash_flow_warnings": "all"  
}  
}
```



```
}
```

Response: {Updated preferences}

WebSocket Endpoints (Real-time updates)

WS /api/v1/ws/document-processing

On connect: {"document_id": "uuid"}

Receive: {

"status": "processing", # or "completed", "failed"

"progress": 75,

"current_step": "Extracting text...",

"result": {...} # When completed

}

WS /api/v1/ws/notifications

Receive: {

"type": "new_notification",

"notification": {...}

}

4.3 WhatsApp Bot API Integration

http

POST /api/v1/whatsapp/webhook

Body: {

"from": "+919876543210",

"message_type": "image|text|document",

"content": "...",

"media_url": "https://..." # For images/docs

}

Response: {Immediate acknowledgment}

Actual processing happens async

Response sent back via WhatsApp Business API

4.4 Internal APIs (Microservices Communication)

http

AI Orchestrator Service

POST /internal/ai/agent-a/audit-document

Body: {
 "document_id": "uuid",
 "file_url": "https://...",
 "user_context": {...}
}

Response: {Audit results}

Document Processing Service

POST /internal/ocr/extract-text

Body: {
 "file_url": "https://...",
 "document_type": "invoice"
}

Response: {
 "extracted_text": "...",
 "structured_data": {...},
 "confidence": 0.92
}

Notification Service

POST /internal/notifications/send

Body: {
 "user_id": "uuid",
 "channel": "whatsapp",
 "message": "...",
 "priority": "high"
}

Response: {"status": "queued"}

5. AI Agent Architecture

5.1 Agent Orchestration Flow

```
python
```

Pseudo-code for AI Orchestrator Service

```
class AIOrchestrator:
```

```
    def __init__(self):
```

```
        self.agent_a = VisualAuditor()
```

```
        self.agent_b = LegislativeSentinel()
```

```
        self.agent_c = SubsidyHunter()
```

```
        self.agent_d = Negotiator()
```

```
    async def process_document(self, document_id, user_id):
```

```
        # Step 1: Agent A - Visual Audit
```

```
        audit_result = await self.agent_a.audit_document(document_id)
```

```
        # Step 2: Store results
```

```
        await db.update_document_audit(document_id, audit_result)
```

```
        # Step 3: If new transaction, check for subsidy eligibility (Agent C)
```

```
        if audit_result.is_compliant and audit_result.transaction_type == "purchase":
```

```
            subsidy_matches = await self.agent_c.find_matching_subsidies(
                user_id,
                audit_result.extracted_data
            )
```

```
            if subsidy_matches:
```

```
                await notification_service.send(
                    user_id,
                    "subsidy_match",
                    subsidy_matches
                )
```

```
        # Step 4: Update compliance monitoring (Agent B)
```

```
        if audit_result.has_compliance_implications:
```

```
            await self.agent_b.update_compliance_calendar(user_id, audit_result)
```

```
return audit_result
```

5.2 Agent A: Visual Auditor Implementation

Tech Stack:

- **Vision Model:** Claude 3.5 Sonnet (multimodal) or GPT-4o
- **OCR:** Google Cloud Vision API (primary), Tesseract (fallback)
- **Validation:** Rule-based + LLM-based checks

Process Flow:

Document Upload → OCR Extraction → LLM Analysis → Rule Validation → Result

Implementation:

```
python
```

```
class VisualAuditor:
    def __init__(self):
        self.ocr_service = GoogleVisionOCR()
        self.llm = AnthropicClient(model="claude-3-5-sonnet-20241022")
        self.validator = ComplianceValidator()

    async def audit_document(self, document_id):
        # Step 1: Get document from storage
        document = await storage.get_document(document_id)

        # Step 2: OCR extraction
        ocr_result = await self.ocr_service.extract_text(document.file_url)

        # Step 3: LLM-based analysis
        prompt = f"""
        Analyze this Indian invoice/bill and extract:
        1. Invoice details (number, date, vendor, customer)
        2. Line items and amounts
        3. GST details (GSTIN, tax amounts)
        4. Compliance issues (missing fields, incorrect format)
        5. Classification (business expense vs personal)

        OCR Text:
        {ocr_result.text}

        Return JSON format with extracted data and compliance verdict.
        """

        llm_response = await self.llm.generate(
            prompt=prompt,
            max_tokens=2000,
            response_format="json"
```

```

)

# Step 4: Rule-based validation
extracted_data = json.loads(llm_response.content)
validation_result = self.validator.validate_invoice(extracted_data)

# Step 5: Combine results
audit_result = {
    "is_compliant": validation_result.is_compliant,
    "compliance_score": validation_result.score,
    "issues": validation_result.issues,
    "extracted_data": extracted_data,
    "confidence": llm_response.confidence,
    "processing_time_ms": time_taken
}

# Step 6: Log to database
await db.insert_agent_log(
    agent_name="agent_a",
    action="audit_invoice",
    input_data={"document_id": document_id},
    output_data=audit_result,
    tokens_used=llm_response.usage.total_tokens,
    cost_usd=calculate_cost(llm_response.usage)
)

return audit_result

```

5.3 Agent B: Legislative Sentinel Implementation

Tech Stack:

- **RAG Framework:** LangChain or LlamaIndex

- **Vector DB:** Pinecone or Weaviate
- **Embedding Model:** text-embedding-3-large (OpenAI)
- **Web Scraper:** Scrapy + BeautifulSoup

Process Flow:

Daily Scrape Gov Websites → Extract PDFs → Chunk Documents → Generate Embeddings
→ Store in Vector DB → User Query → Semantic Search → Return Relevant Updates

Implementation:

```
python
```

```
class LegislativeSentinel:
```

```
    def __init__(self):
```

```
        self.vector_db = PineconeClient(index_name="legislative_docs")
```

```
        self.embedder = OpenAIEmbeddings(model="text-embedding-3-large")
```

```
        self.llm = AnthropicClient()
```

```
    async def monitor_updates(self):
```

```
        """
```

```
        Cron job: Runs daily to scrape gov websites
```

```
        """
```

```
        # Sources to scrape
```

```
        sources = [
```

```
            "https://cbic-gst.gov.in/", # GST updates
```

```
            "https://www.mca.gov.in/", # MCA circulars
```

```
            # ... other sources
```

```
        ]
```

```
        new_documents = []
```

```
        for source in sources:
```

```
            docs = await self.scrapers.fetch_latest(source)
```

```
            new_documents.extend(docs)
```

```
        # Process and embed new documents
```

```
        for doc in new_documents:
```

```
            chunks = self.chunk_document(doc.content, chunk_size=500)
```

```
            for chunk in chunks:
```

```
                embedding = await self.embedder.embed(chunk.text)
```

```
            await self.vector_db.upsert(
```

```
                id=f"{doc.id}_{chunk.index}",
```

```
                vector=embedding,
```

```

        metadata={
            "title": doc.title,
            "source": doc.source,
            "date": doc.published_date,
            "chunk_text": chunk.text
        }
    )

    # Notify affected users
    await self.notify_affected_users(new_documents)

    async def check_user_relevance(self, user_id, update):
        """
        Check if a legislative update affects this user
        """
        user = await db.get_user(user_id)

        # Semantic search to check relevance
        user_profile_text = f"""
        Business Type: {user.business_type}
        Industry: {user.industry_sector}
        State: {user.state}
        Revenue: {user.revenue_range}
        """

        profile_embedding = await self.embedder.embed(user_profile_text)
        update_embedding = await self.embedder.embed(update.summary)

        similarity = cosine_similarity(profile_embedding, update_embedding)

        if similarity > 0.7: # Threshold
            # Generate user-specific summary

```

```

summary = await self.llm.generate(
    prompt=f"""
    Given this legislative update:
    {update.full_text}

    And this user profile:
    {user_profile_text}

    Explain in 2-3 sentences how this update specifically affects this user.
    """
)

return {
    "is_relevant": True,
    "impact_summary": summary.content,
    "similarity_score": similarity
}

return {"is_relevant": False}

```

5.4 Agent C: Subsidy Hunter Implementation

Tech Stack:

- **Embedding Model:** text-embedding-3-large
- **Vector DB:** Pinecone (subsidy index)
- **Matching Algorithm:** Semantic search + Rule-based filters

Process Flow:

User Transaction → Extract Context → Generate Query Embedding → Vector Search
→ Filter by Eligibility Rules → Rank by Match Score → Return Top Matches

Implementation:

```
python
```

```
class SubsidyHunter:
```

```
    def __init__(self):
```

```
        self.vector_db = PineconeClient(index_name="subsidies")
```

```
        self.embedder = OpenAIEmbeddings()
```

```
        self.llm = AnthropicClient()
```

```
    async def find_matching_subsidies(self, user_id, transaction_data):
```

```
        """
```

```
        Find subsidies matching user's recent transaction/profile
```

```
        """
```

```
        # Step 1: Get user context
```

```
        user = await db.get_user(user_id)
```

```
        recent_transactions = await db.get_recent_transactions(user_id, days=30)
```

```
        # Step 2: Build search query
```

```
        search_context = f"""
```

```
Recent Purchase: {transaction_data.get('description')}
```

```
Amount: {transaction_data.get('amount')}
```

```
Category: {transaction_data.get('category')}
```

```
Business Type: {user.business_type}
```

```
State: {user.state}
```

```
Revenue Range: {user.revenue_range}
```

```
        """
```

```
        # Step 3: Semantic search
```

```
        query_embedding = await self.embedder.embed(search_context)
```

```
        search_results = await self.vector_db.query(
```

```
            vector=query_embedding,
```

```
            top_k=20,
```

```
            filter={
```

```
                "is_active": True,
```

```
        "state_specific": {"$in": [None, user.state]}
    }
)
```

Step 4: Rule-based filtering + Detailed matching

```
matches = []
for result in search_results:
    subsidy = await db.get_subsidy(result.id)

    # Check eligibility criteria
    eligibility_check = await self.check_eligibility(
        user,
        transaction_data,
        subsidy.eligibility_criteria
    )

    if eligibility_check.is_eligible:
        matches.append({
            "subsidy_id": subsidy.id,
            "scheme_name": subsidy.scheme_name,
            "match_score": result.score * 100, # 0-100
            "max_benefit": subsidy.max_benefit_amount,
            "match_reason": eligibility_check.reason,
            "deadline": subsidy.deadline
        })
```

Step 5: Store matches in database

```
for match in matches:
    await db.insert_subsidy_match(
        user_id=user_id,
        subsidy_id=match['subsidy_id'],
        match_score=match['match_score'],
```

```

        match_reason=match['match_reason']
    )

    return sorted(matches, key=lambda x: x['match_score'], reverse=True)[:5]

async def generate_application_draft(self, user_id, subsidy_id):
    """
    Auto-generate subsidy application
    """
    user = await db.get_user(user_id)
    subsidy = await db.get_subsidy(subsidy_id)
    user_documents = await db.get_user_documents(user_id)

    prompt = f"""
    You are filling out a government subsidy application form.

    Subsidy Scheme: {subsidy.scheme_name}
    Required Information: {subsidy.eligibility_criteria}

    Applicant Details:
    - Business Name: {user.business_name}
    - GSTIN: {user.gstin}
    - State: {user.state}
    - Industry: {user.industry_sector}

    Generate a complete application draft in JSON format with all required fields filled.
    For missing information, indicate with "NEEDS_USER_INPUT".
    """

    draft = await self.llm.generate(
        prompt=prompt,
        max_tokens=3000,

```



```
        response_format="json"  
    )  
  
    return json.loads(draft.content)
```

5.5 Agent D: Negotiator Implementation

Tech Stack:

- **LLM:** Claude Sonnet 4.5 (fine-tuned on business correspondence)
- **Email Service:** SendGrid or AWS SES
- **Templates:** Pre-written business email templates

Process Flow:

Cash Flow Alert → Identify Overdue Invoices → Generate Email Draft →
User Review/Edit → Send Email → Track Response

Implementation:

python

```
class Negotiator:
    def __init__(self):
        self.llm = AnthropicClient(model="claude-sonnet-4-5-20250929")
        self.email_service = SendGridClient()

    async def draft_collection_email(self, invoice_id, tone="polite"):
        """
        Generate payment collection email
        """
        invoice = await db.get_invoice(invoice_id)
        user = await db.get_user(invoice.user_id)

        tone_instructions = {
            "polite": "friendly and understanding, emphasizing partnership",
            "firm": "professional but assertive, mentioning payment terms",
            "urgent": "direct and clear about consequences, while remaining professional"
        }

        prompt = f"""
        You are a financial assistant drafting a payment reminder email.

        Context:
        - Your company: {user.business_name}
        - Customer: {invoice.customer_name}
        - Invoice Number: {invoice.invoice_number}
        - Amount Due: ₹{invoice.total_amount}
        - Days Overdue: {invoice.days_overdue}
        - Original Due Date: {invoice.payment_due_date}

        Tone: {tone_instructions[tone]}

        Write a professional email with:
```

1. Subject line
2. Greeting
3. Payment reminder (specific invoice details)
4. Request for payment or response
5. Closing

Keep it concise (under 200 words). Use Indian business English conventions.
Return as JSON with "subject" and "body" fields.

```
"""
```

```
email_draft = await self.llm.generate(  
    prompt=prompt,  
    max_tokens=800,  
    response_format="json"  
)
```

```
return json.loads(email_draft.content)
```

```
async def send_email(self, email_draft_id, user_edits=None):
```

```
    """
```

```
    Send the drafted email
```

```
    """
```

```
    draft = await db.get_email_draft(email_draft_id)
```

```
    if user_edits:
```

```
        draft.update(user_edits)
```

```
    result = await self.email_service.send(  
        to=draft['to_email'],
```

```
        subject=draft['subject'],
```

```
        body=draft['body'],
```

```
        from_email=f"{user.business_name} <noreply@micro-cfo.com>"
```

```
)

# Log the action
await db.insert_agent_log(
    agent_name="agent_d",
    action="send_collection_email",
    output_data={"email_id": result.id, "status": result.status}
)

return result
```

6. Third-Party Integrations

6.1 WhatsApp Business API Integration

Provider: Twilio or Meta WhatsApp Business Platform

Implementation:

```
python
```

```
class WhatsAppService:
    def __init__(self):
        self.client = TwilioClient(
            account_sid=config.TWILIO_SID,
            auth_token=config.TWILIO_TOKEN
        )
        self.whatsapp_number = config.WHATSAPP_NUMBER

    async def handle_incoming_message(self, webhook_data):
        """
        Handle incoming WhatsApp messages
        """
        from_number = webhook_data['from']
        message_type = webhook_data['type'] # text, image, document

        # Get or create user
        user = await db.get_user_by_phone(from_number)
        if not user:
            # New user onboarding flow
            await self.start_onboarding(from_number)
            return

        if message_type == 'image':
            # Document upload via WhatsApp
            media_url = webhook_data['media_url']

            # Download and upload to S3
            file_url = await storage.upload_from_url(media_url)

            # Process document
            document = await db.create_document(
                user_id=user.id,
```

```

        file_url=file_url,
        uploaded_via='whatsapp'
    )

    # Queue for AI processing
    await queue.publish('document.processing', {
        'document_id': document.id,
        'user_id': user.id
    })

    # Send acknowledgment
    await self.send_message(
        from_number,
        " 📄 Document received! Analyzing now... You'll get results in 30 seconds."
    )

    elif message_type == 'text':
        # Handle text commands
        await self.handle_text_command(user, webhook_data['text'])

    async def send_message(self, to_number, message):
        """
        Send WhatsApp message
        """
        self.client.messages.create(
            from_=f'whatsapp:{self.whatsapp_number}',
            to=f'whatsapp:{to_number}',
            body=message
        )

    async def send_audit_result(self, user_id, audit_result):
        """

```

Send document audit results via WhatsApp

```
"""
```

```
user = await db.get_user(user_id)
```

```
if audit_result['is_compliant']:
```

```
    message = f"✅ Document Verified!\n\nAmount: ₹ {audit_result['amount']}\nVendor: {audit_result['vendor']}\n"
```

```
else:
```

```
    issues = "\n".join([f"• {issue}" for issue in audit_result['issues']])
```

```
    message = f"⚠️ Compliance Alert!\n\n{issues}\n\nDo NOT file this without corrections."
```

```
await self.send_message(user.phone_number, message)
```

6.2 Account Aggregator Framework Integration

Provider: Sahamati-certified AA providers (FinVu, OneMoney, etc.)

Implementation:

```
python
```

```
class AccountAggregatorService:
    def __init__(self):
        self.aa_client = AAClient(
            client_id=config.AA_CLIENT_ID,
            client_secret=config.AA_CLIENT_SECRET
        )

    async def request_consent(self, user_id, bank_accounts):
        """
        Request user consent to access bank data
        """
        consent_request = await self.aa_client.create_consent(
            user_id=user_id,
            fi_types=['DEPOSIT'], # Bank accounts
            data_range={
                'from': '2024-01-01',
                'to': '2024-12-31'
            },
            data_life={'value': 1, 'unit': 'YEAR'},
            frequency={'value': 1, 'unit': 'DAILY'}
        )

        # Store consent ID
        await db.update_user(user_id, {
            'aa_consent_id': consent_request.id
        })

        # Return consent URL for user approval
        return consent_request.consent_handle_url

    async def fetch_bank_data(self, user_id):
        """
```


Fetch bank statements via AA framework

```
"""
```

```
user = await db.get_user(user_id)
```

```
if not user.aa_consent_id:
```

```
    raise Exception("No AA consent found")
```

```
# Fetch financial information
```

```
fi_data = await self.aa_client.fetch_data(  
    consent_id=user.aa_consent_id  
)
```

```
# Parse and store transactions
```

```
transactions = self.parse-fi_data(fi_data)
```

```
await db.store_aa_data(  
    user_id=user_id,
```

```
    transactions=transactions,
```

```
    balance=fi_data.current_balance
```

```
)
```

```
return transactions
```

6.3 Government Portal Scrapers

Implementation:

```
python
```

```
class GovernmentDataScraper:
```

```
    def __init__(self):
```

```
        self.sources = {
```

```
            'gst': 'https://cbic-gst.gov.in/',
```

```
            'mca': 'https://www.mca.gov.in/',
```

```
            'msme': 'https://msme.gov.in/',
```

```
            'startup_india': 'https://www.startupindia.gov.in/'
```

```
        }
```

```
    async def scrape_subsidies(self):
```

```
        """
```

```
        Daily job to scrape government subsidy schemes
```

```
        """
```

```
    async with aiohttp.ClientSession() as session:
```

```
        for source_name, url in self.sources.items():
```

```
            html = await self.fetch_page(session, url)
```

```
            # Extract subsidy announcements
```

```
            subsidies = self.parse_subsidies(html, source_name)
```

```
        for subsidy in subsidies:
```

```
            # Check if already exists
```

```
            existing = await db.get_subsidy_by_code(subsidy.scheme_code)
```

```
            if not existing:
```

```
                # Generate embedding
```

```
                embedding = await embedder.embed(subsidy.description)
```

```
                # Store in database
```

```
                await db.insert_subsidy({
```

```
                    ##subsidy,
```

```
                    'embedding': embedding
```

```
})
```

```
async def scrape_legislative_updates(self):  
    """  
    Daily job to scrape new laws/circulars  
    """  
    # Similar to scrape_subsidies  
    # Store in legislative_updates table + vector DB  
    pass
```

6.4 Email Integration (SendGrid/AWS SES)

Implementation:

```
python
```

```
class EmailService:
    def __init__(self):
        self.sendgrid = SendGridAPIClient(config.SENDGRID_API_KEY)

    async def send_email(self, to, subject, body, from_name):
        """
        Send transactional email
        """
        message = Mail(
            from_email=Email(f'noreply@micro-cfo.com', from_name),
            to_emails=To(to),
            subject=subject,
            html_content=body
        )

        try:
            response = self.sendgrid.send(message)

            # Log email sent
            await db.insert_notification_log({
                'channel': 'email',
                'to': to,
                'subject': subject,
                'status': 'sent',
                'provider_response': response.status_code
            })

            return response
        except Exception as e:
            logger.error(f'Email send failed: {e}')
            raise
```

7. Authentication & Security

7.1 Authentication Flow

Primary Method: Phone Number + OTP

```
python
```

```
class AuthService:
    def __init__(self):
        self.otp_service = TwilioOTP()
        self.jwt_secret = config.JWT_SECRET

    async def send_otp(self, phone_number):
        """
        Send OTP for login/registration
        """
        # Generate 6-digit OTP
        otp = self.generate_otp()

        # Store in Redis with 5-minute expiry
        await redis.setex(
            f"otp:{phone_number}",
            300, # 5 minutes
            otp
        )

        # Send via SMS
        await self.otp_service.send(phone_number, otp)

    async def verify_otp(self, phone_number, otp):
        """
        Verify OTP and issue JWT
        """
        stored_otp = await redis.get(f"otp:{phone_number}")

        if not stored_otp or stored_otp != otp:
            raise InvalidOTPError()

        # Delete OTP after verification
```

```
await redis.delete(f'otp:{phone_number}')
```

```
# Get or create user
```

```
user = await db.get_user_by_phone(phone_number)
```

```
if not user:
```

```
    user = await db.create_user({'phone_number': phone_number})
```

```
# Generate JWT tokens
```

```
access_token = self.create_access_token(user.id)
```

```
refresh_token = self.create_refresh_token(user.id)
```

```
# Store refresh token
```

```
await redis.setex(
```

```
    f'refresh_token:{user.id}',
```

```
    86400 * 30, # 30 days
```

```
    refresh_token
```

```
)
```

```
return {
```

```
    'access_token': access_token,
```

```
    'refresh_token': refresh_token,
```

```
    'user': user
```

```
}
```

```
def create_access_token(self, user_id):
```

```
    """
```

```
    Create JWT access token (24h expiry)
```

```
    """
```

```
    payload = {
```

```
        'user_id': str(user_id),
```

```
        'type': 'access',
```

```
        'exp': datetime.utcnow() + timedelta(hours=24)
```

```
}  
return jwt.encode(payload, self.jwt_secret, algorithm='HS256')  
  
def create_refresh_token(self, user_id):  
    """  
    Create JWT refresh token (30d expiry)  
    """  
    payload = {  
        'user_id': str(user_id),  
        'type': 'refresh',  
        'exp': datetime.utcnow() + timedelta(days=30)  
    }  
    return jwt.encode(payload, self.jwt_secret, algorithm='HS256')
```

7.2 Security Measures

Data Encryption:

python


```

from cryptography.fernet import Fernet

class EncryptionService:
    def __init__(self):
        self.cipher = Fernet(config.ENCRIPTION_KEY)

    def encrypt(self, plaintext):
        return self.cipher.encrypt(plaintext.encode()).decode()

    def decrypt(self, ciphertext):
        return self.cipher.decrypt(ciphertext.encode()).decode()

# Encrypt sensitive fields before storing
encrypted_account = encryption.encrypt(user.bank_account_number)

```

Rate Limiting:

```

python

from fastapi_limiter import FastAPILimiter
from fastapi_limiter.depends import RateLimiter

@app.post("/api/v1/auth/send-otp")
@limiter.limit("5/minute") # Max 5 OTP requests per minute
async def send_otp(phone: str):
    # ...

```

Input Validation:

```

python

```

```
from pydantic import BaseModel, validator

class DocumentUpload(BaseModel):
    document_type: str

    @validator('document_type')
    def validate_type(cls, v):
        allowed_types = ['invoice', 'bill', 'receipt', 'contract']
        if v not in allowed_types:
            raise ValueError(f'Invalid type. Must be one of {allowed_types}')
        return v
```

API Key Management:

- Store all API keys in environment variables
- Use AWS Secrets Manager or HashiCorp Vault in production
- Rotate keys quarterly
- Different keys for dev/staging/production

HTTPS Only:

- All API endpoints must use HTTPS
 - Implement HSTS headers
 - Certificate pinning for mobile apps
-

8. Scalability & Performance

8.1 Database Optimization

Indexing Strategy:

```
sql

-- High-traffic query indexes
CREATE INDEX CONCURRENTLY idx_documents_user_status
ON documents(user_id, processing_status)
WHERE processing_status IN ('pending', 'processing');

CREATE INDEX CONCURRENTLY idx_invoices_overdue
ON invoices(user_id, payment_due_date)
WHERE payment_status = 'pending' AND payment_due_date < CURRENT_DATE;

-- Partial indexes for common filters
CREATE INDEX idx_subsidies_active_sector
ON subsidies(sector_applicable)
WHERE is_active = TRUE;
```

Connection Pooling:

```
python
```

```
from sqlalchemy import create_engine
from sqlalchemy.pool import QueuePool

engine = create_engine(
    DATABASE_URL,
    poolclass=QueuePool,
    pool_size=20,
    max_overflow=40,
    pool_pre_ping=True # Verify connections before use
)
```

Read Replicas:

```
python

# Write to primary
await primary_db.insert_document(...)

# Read from replica
documents = await replica_db.get_user_documents(user_id)
```

8.2 Caching Strategy

Multi-Level Caching:

```
python
```

```
class CacheService:
    def __init__(self):
        self.redis = RedisClient()
        self.local_cache = {} # In-memory cache

    async def get_user(self, user_id):
        # L1: In-memory cache (fastest)
        if user_id in self.local_cache:
            return self.local_cache[user_id]

        # L2: Redis cache
        cached = await self.redis.get(f'user:{user_id}')
        if cached:
            user = json.loads(cached)
            self.local_cache[user_id] = user
            return user

        # L3: Database
        user = await db.get_user(user_id)

        # Update caches
        await self.redis.setex(f'user:{user_id}', 3600, json.dumps(user))
        self.local_cache[user_id] = user

        return user
```

Cache Invalidation:

```
python
```

```
# Invalidate on update
async def update_user(user_id, updates):
    await db.update_user(user_id, updates)

# Clear caches
await redis.delete(f'user:{user_id}')
local_cache.pop(user_id, None)
```

8.3 Asynchronous Processing

Task Queue Architecture:

```
python
```

```

from celery import Celery

celery_app = Celery('micro_cfo', broker='redis://localhost:6379/0')

@celery_app.task
def process_document(document_id):
    """
    Long-running task for document processing
    """
    # This runs in background worker
    result = ai_orchestrator.process_document(document_id)

    # Send notification when complete
    notification_service.send(...)

# Queue task from API
@app.post("/documents/upload")
async def upload_document(...):
    document = await db.create_document(...)

    # Queue for background processing
    process_document.delay(document.id)

    return {"status": "queued", "document_id": document.id}

```

8.4 Horizontal Scaling

Stateless API Servers:

- No session state stored in memory
- JWT tokens for authentication (no server-side sessions)

- All state in database/cache

Load Balancing:

```
yaml

# Kubernetes deployment example
apiVersion: apps/v1
kind: Deployment
metadata:
  name: micro-cfo-api
spec:
  replicas: 5 # Scale to 5 instances
  selector:
    matchLabels:
      app: micro-cfo-api
  template:
    metadata:
      labels:
        app: micro-cfo-api
    spec:
      containers:
        - name: api
          image: micro-cfo-api:latest
          resources:
            limits:
              memory: "2Gi"
              cpu: "1000m"
```

Auto-scaling Rules:

- CPU > 70% for 5 minutes → Scale up
- CPU < 30% for 10 minutes → Scale down

- Min replicas: 2, Max replicas: 20

9. Deployment & DevOps

9.1 Infrastructure as Code (Terraform)

hcl

```
# Example Terraform configuration

resource "aws_ecs_cluster" "micro_cfo" {
  name = "micro-cfo-cluster"
}

resource "aws_ecs_service" "api" {
  name           = "micro-cfo-api"
  cluster        = aws_ecs_cluster.micro_cfo.id
  task_definition = aws_ecs_task_definition.api.arn
  desired_count  = 3

  load_balancer {
    target_group_arn = aws_lb_target_group.api.arn
    container_name   = "api"
    container_port    = 8000
  }
}

resource "aws_rds_instance" "postgres" {
  identifier      = "micro-cfo-db"
  engine          = "postgres"
  engine_version  = "15.3"
  instance_class  = "db.t3.medium"
  allocated_storage = 100
  storage_encrypted = true
  multi_az         = true
  backup_retention_period = 7
}
```

9.2 CI/CD Pipeline (GitHub Actions)

yaml

```
#.github/workflows/deploy.yml
```

```
name: Deploy to Production
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v2
```

```
      - name: Run tests
```

```
        run: |
```

```
          pip install -r requirements.txt
```

```
          pytest tests/
```

```
  build:
```

```
    needs: test
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Build Docker image
```

```
        run: docker build -t micro-cfo-api:${{ github.sha }} .
```

```
      - name: Push to ECR
```

```
        run: |
```

```
          aws ecr get-login-password | docker login --username AWS --password-stdin $ECR_REGISTRY
```

```
          docker push micro-cfo-api:${{ github.sha }}
```

```
  deploy:
```

```
    needs: build
```

```
    runs-on: ubuntu-latest
```

steps:

- name: Deploy to ECS

run: |

```
aws ecs update-service --cluster micro-cfo --service api --force-new-deployment
```

9.3 Environment Configuration

python

```
# config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    # Database
    DATABASE_URL: str
    DATABASE_REPLICA_URL: str

    # Redis
    REDIS_URL: str

    # APIs
    ANTHROPIC_API_KEY: str
    OPENAI_API_KEY: str
    WHATSAPP_API_KEY: str
    SENDGRID_API_KEY: str

    # S3
    AWS_ACCESS_KEY: str
    AWS_SECRET_KEY: str
    S3_BUCKET_NAME: str

    # JWT
    JWT_SECRET: str
    ENCRYPTION_KEY: str

    # Feature Flags
    ENABLE_AGENT_B: bool = True
    ENABLE_AGENT_C: bool = True

class Config:
    env_file = ".env"
```

```
settings = Settings()
```

9.4 Monitoring & Alerts

Prometheus Metrics:

```
python

from prometheus_client import Counter, Histogram

# Custom metrics
document_uploads = Counter('document_uploads_total', 'Total document uploads')
api_request_duration = Histogram('api_request_duration_seconds', 'API request duration')

@app.post("/documents/upload")
async def upload_document(...):
    document_uploads.inc()

    with api_request_duration.time():
        # ... processing
    pass
```

Health Checks:

```
python
```

```
@app.get("/health")
async def health_check():
    checks = {
        "database": await check_db_connection(),
        "redis": await check_redis_connection(),
        "s3": await check_s3_access(),
        "openai_api": await check_openai_api()
    }

    status = "healthy" if all(checks.values()) else "unhealthy"

    return {
        "status": status,
        "checks": checks,
        "timestamp": datetime.utcnow()
    }
```

10. Monitoring & Observability

10.1 Logging Strategy

Structured Logging:

```
python
```



```
import structlog

logger = structlog.get_logger()

logger.info(
    "document_processed",
    document_id=document.id,
    user_id=user.id,
    processing_time_ms=execution_time,
    compliance_status=result.is_compliant,
    agent="agent_a"
)
```

Log Aggregation (ELK Stack):

- Elasticsearch: Store logs
- Logstash: Process and transform logs
- Kibana: Visualize and search logs

10.2 APM (Application Performance Monitoring)

Datadog Integration:

```
python

from ddtrace import tracer

@tracer.wrap(service="micro-cfo-api", resource="process_document")
async def process_document(document_id):
    # Automatically tracked
    pass
```

10.3 Error Tracking (Sentry)

```
python

import sentry_sdk

sentry_sdk.init(
    dsn=config.SENTRY_DSN,
    environment="production",
    traces_sample_rate=0.1
)

# Errors automatically captured
try:
    result = await ai_agent.process()
except Exception as e:
    sentry_sdk.capture_exception(e)
    raise
```

10.4 Business Metrics Dashboard

Key Metrics to Track:

```
python
```

Daily metrics

- Total documents processed
- Average processing time
- Compliance issue detection rate
- Subsidy matches found
- Emails sent by Agent D
- User retention rate (DAU/MAU)

Financial metrics

- API costs (OpenAI, Anthropic)
- Infrastructure costs
- Revenue per user

Agent performance

- Agent A accuracy (based on user feedback)
- Agent C conversion rate (subsidy applications)
- Agent D email response rate

11. Data Pipeline & ETL

11.1 Daily ETL Jobs

Job 1: Government Data Scraping

```
python
```

```
# Airflow DAG
from airflow import DAG
from airflow.operators.python import PythonOperator

dag = DAG(
    'scrape_government_data',
    schedule_interval='@daily',
    start_date=datetime(2024, 1, 1)
)

scrape_subsidies_task = PythonOperator(
    task_id='scrape_subsidies',
    python_callable=scraper.scrape_subsidies,
    dag=dag
)

scrape_laws_task = PythonOperator(
    task_id='scrape_legislative_updates',
    python_callable=scraper.scrape_legislative_updates,
    dag=dag
)

scrape_subsidies_task >> scrape_laws_task
```

Job 2: Compliance Calendar Updates

```
python
```

```
async def update_compliance_calendars():  
    """  
    Daily job: Update all users' compliance calendars  
    """  
    users = await db.get_all_active_users()  
  
    for user in users:  
        # Calculate upcoming deadlines based on business type  
        deadlines = calculate_deadlines(user)  
  
        for deadline in deadlines:  
            await db.upsert_compliance_event(user.id, deadline)
```

Job 3: Cash Flow Predictions

```
python
```

```

async def generate_cash_flow_predictions():
    """
    Daily job: Generate 90-day cash flow forecasts
    """
    users = await db.get_all_active_users()

    for user in users:
        transactions = await db.get_user_transactions(user.id, days=180)
        predictions = ml_model.predict_cash_flow(transactions)

        await db.store_predictions(user.id, predictions)

    # Check for critical alerts
    if predictions.has_shortfall:
        await notification_service.send_alert(user.id, predictions)

```

11.2 Data Backup Strategy

PostgreSQL Backups:

```

bash

# Automated daily backups
0 2 * * * pg_dump -h $DB_HOST -U $DB_USER micro_cfo_db | gzip > /backups/db_$(date +%Y%m%d).sql.gz

# Retention: 30 days

```

S3 Versioning:

- Enable versioning on S3 bucket
- Lifecycle policy: Retain for 1 year, then archive to Glacier

12. Development Timeline

Phase 1: Foundation (Weeks 1-4)

Week 1-2: Infrastructure Setup

- Set up AWS/GCP account
- Provision PostgreSQL, Redis, S3
- Set up development environment
- Initialize Git repository
- Create database schemas

Week 3-4: Core API Development

- Authentication service (OTP, JWT)
- User management APIs
- Document upload API
- Basic CRUD operations
- API documentation (Swagger)

Phase 2: AI Agent Development (Weeks 5-8)

Week 5: Agent A (Visual Auditor)

- OCR integration (Google Vision)
- LLM integration (Claude/GPT-4)
- Document extraction pipeline
- Compliance validation logic

Week 6: Agent B (Legislative Sentinel)

- Web scraper setup
- Vector database setup
- RAG pipeline implementation
- Legislative update monitoring

Week 7: Agent C (Subsidy Hunter)

- Subsidy database seeding
- Semantic search implementation
- Matching algorithm
- Application draft generation

Week 8: Agent D (Negotiator)

- Email template creation
- LLM fine-tuning for business emails
- SendGrid integration
- Email tracking

Phase 3: Integrations (Weeks 9-10)

Week 9: WhatsApp Integration

- Twilio/Meta setup
- Webhook handling
- Message routing

- Image/document processing via WhatsApp

Week 10: Account Aggregator

- AA framework integration
- Bank data fetching
- Transaction parsing
- Cash flow analysis

Phase 4: Testing & Optimization (Weeks 11-12)

Week 11: Testing

- Unit tests (80% coverage)
- Integration tests
- Load testing (JMeter/Locust)
- Security testing

Week 12: Optimization

- Performance optimization
- Caching implementation
- Database query optimization
- Final bug fixes

Phase 5: Deployment (Week 13-14)

Week 13: Production Setup

- Production infrastructure provisioning

- CI/CD pipeline setup
- Monitoring and logging setup
- Security hardening

Week 14: Soft Launch

- Deploy to production
- Beta user testing (50 users)
- Bug fixes
- Performance monitoring

Post-Launch (Week 15+)

- User feedback incorporation
- Feature iterations
- Scaling based on load
- Continuous improvements

13. Cost Estimation (Monthly)

Infrastructure Costs (AWS)

EC2/ECS (API servers): \$300

RDS PostgreSQL (db.t3.medium): \$150

Redis (ElastiCache): \$100

S3 Storage (1TB documents): \$30

CloudFront CDN: \$50
Total Infrastructure: ~\$630/month

Third-Party Services

Anthropic API (500K tokens/day): \$300
OpenAI API (backup/embeddings): \$200
WhatsApp Business API (10K users): \$500
SendGrid (Email): \$100
Google Cloud Vision (OCR): \$150
Pinecone (Vector DB): \$100
Monitoring (Datadog): \$200
Total Services: ~\$1,550/month

Total Estimated Cost

For 10,000 active users: ~\$2,200/month
Per-user cost: \$0.22/month

14. Security Checklist

- ☐ All API endpoints use HTTPS
- ☐ JWT tokens expire in 24 hours
- ☐ Sensitive data encrypted at rest
- ☐ Rate limiting on all endpoints
- ☐ Input validation on all user inputs
- ☐ SQL injection prevention (parameterized queries)
- ☐ XSS protection

- ☐ CORS properly configured
 - ☐ API keys stored in secrets manager
 - ☐ Regular security audits scheduled
 - ☐ DDoS protection (CloudFlare)
 - ☐ Database backups automated
 - ☐ Disaster recovery plan documented
-

15. Testing Strategy

Unit Tests

```
python
```

```
# tests/test_agent_a.py
import pytest

@pytest.mark.asyncio
async def test_visual_auditor_valid_invoice():
    auditor = VisualAuditor()
    result = await auditor.audit_document(sample_invoice_id)

    assert result['is_compliant'] == True
    assert result['extracted_data']['total_amount'] == 50000
    assert len(result['issues']) == 0

@pytest.mark.asyncio
async def test_visual_auditor_personal_expense():
    auditor = VisualAuditor()
    result = await auditor.audit_document(personal_bill_id)

    assert result['is_compliant'] == False
    assert 'personal expense' in result['issues'][0].lower()
```

Integration Tests

```
python
```

```
@pytest.mark.asyncio
async def test_document_upload_to_audit_flow():
    # Upload document
    response = await client.post(
        "/api/v1/documents/upload",
        files={"file": open("test_invoice.jpg", "rb")}
    )
    document_id = response.json()['document_id']

    # Wait for processing
    await asyncio.sleep(5)

    # Check audit result
    result = await client.get(f"/api/v1/documents/{document_id}")
    assert result.json()['processing_status'] == 'completed'
```

Load Tests (Locust)

python

```
from locust import HttpUser, task, between
```

```
class MicroCFOUser(HttpUser):
```

```
    wait_time = between(1, 3)
```

```
    @task
```

```
    def upload_document(self):
```

```
        with open("test_invoice.jpg", "rb") as f:
```

```
            self.client.post(
```

```
                "/api/v1/documents/upload",
```

```
                files={"file": f}
```

```
            )
```

```
    @task(3) # 3x more frequent
```

```
    def get_dashboard(self):
```

```
        self.client.get("/api/v1/dashboard")
```

16. Documentation Requirements

API Documentation

- Auto-generated OpenAPI/Swagger docs
- Postman collection for all endpoints
- Code examples in Python, JavaScript, cURL

Developer Docs

- Architecture overview
- Database schema documentation

- Setup instructions (local development)
- Deployment guide
- Troubleshooting guide

Operational Runbooks

- Incident response procedures
 - Database migration process
 - Backup and restore procedures
 - Monitoring and alerting setup
-

Conclusion

This backend PRD provides a comprehensive blueprint for building the Micro-CFO system. The architecture is designed for:

- ✅ **Scalability** - Microservices, message queues, horizontal scaling
- ✅ **Performance** - Caching, async processing, optimized queries
- ✅ **Reliability** - Health checks, error handling, monitoring
- ✅ **Security** - Encryption, authentication, rate limiting
- ✅ **Maintainability** - Clean code, comprehensive tests, documentation

Recommended Execution Order:

1. Set up infrastructure (databases, storage, Redis)
2. Build core API server with authentication
3. Develop Agent A (most critical for MVP)
4. Add WhatsApp integration

5. Implement Agents B, C, D in parallel
6. Integrate external services
7. Comprehensive testing
8. Deploy to production

Key Success Factors:

- Start with Agent A for immediate value
- Use managed services (RDS, ElastiCache) to reduce operational burden
- Implement observability from day one
- Keep initial scope tight, iterate based on user feedback
- Monitor AI API costs closely (can spike quickly)

Good luck building the backend! 🚀